QN-MHP Python
WORK INSTRUCTION

Contents

1. System Overview 1
2. System architecture 1
3. User operating procedure 1
3.1 Start the program 1
3.2 Define tasks 1
3.3 Define entities and BE parameters 1
3.4 Define simulation parameters 2
4. File-by-file reference 2
5. Basic BE parameter reference 3
6. Compound BE parameter reference 4
7. How each basic BE operates 5
8. How each compound BE operates 5
9. Displays and plotting 7
10. Timing, units 7
11. Practical troubleshooting notes 8
12. An Example 8
12.1 example overview 8
12.2 start the program 8
12.3 Define task 9
12.4 Define entity and BE parameters 10
12.5 Define simulation parameters 12
12.6 Animation and reaction displays 12
12.7 Plot and log 13

1. System Overview

The QN-MHP system simulates human perception, cognition, and motor behavior through a queueing network structure implemented in SimPy. It supports both basic behavior elements (Basic BEs), which define the basic actions involved in a task, and compound behavior elements (Compound BEs), which define complete tasks composed of multiple Basic BEs.

2. System architecture

The project combines four layers: configuration GUI, entity generation, behavior execution, and result display.

GUI setup	Entity generation	BE execution	Plots and summaries

Note: This program is primarily intended as a demonstration platform for illustrating the structure and behavior of QN-MHP behavior elements (BEs). For future developers, the current implementation can also serve as a modular library of BE functions and classes. If only a subset of BEs is needed for a more complex or task-specific application, developers can directly call the relevant BE functions or classes, adjust their parameters, and recombine them to build new task flows beyond the predefined GUI configurations. This design makes the codebase suitable not only for demonstration purposes, but also as a foundation for further model development and customized simulation tasks.

3. User operating procedure

3.1 Start the program

Open a terminal in the project directory and run `run_code.py`.

The main menu window appears with four primary actions: open, save as, define task, define entity and BE parameters, define simulation parameters, and start.

The system uses a temporary folder named backup/ to store task definitions, entity settings, and BE parameters during the current session. This folder is automatically cleared when the program starts, so configurations are not preserved by default. The Save As function should be used to save the current configuration to a user-selected folder for future reuse. The Open function loads a previously saved configuration folder and restores the stored settings in the GUI. Therefore, if Save As is not used, all current settings will be lost after the program is closed.

3.2 Define tasks

Click `1. Define Task` to open the task-definition matrix.

Row 1 selects the task numbers that are active. Up to five tasks are supported by the UI structure. But currently the project only supports up to 2 tasks running simultaneously.

Rows 2-17 define the behavior-element sequence for each task. In each task column pair, the odd column stores the BE name and the even column stores its order index.

The available BE menu includes both basic and compound BEs. The order values determine execution sequence.

Click `Save and Back to Main Menu` after the task structure is finished.

3.3 Define entities and BE parameters

Click `2. Define Entity and BE Parameters` to open the task-by-task parameter launcher.

The window shows one vertical frame per active task.

Rows without additional parameters are shown as plain labels. Rows that require extra settings become buttons that open the BE-specific dialog.

3.4 Define simulation parameters

Click `3. Define Simulation Parameters`.

Enter total simulation time in milliseconds.

Note: All time values in this system are represented in SimPy simulation time, which is intended to model time progression in the real world. However, the actual program execution time is not the same as simulated time. The wall-clock time required to run the program depends on factors such as CPU performance, system load, and whether animation is enabled. Therefore, simulation time should be interpreted as modeled real-world time, while the actual runtime of the software is determined by computational conditions.

A simulation will be terminated once a task is completed or the simulation time is reached.

Choose whether animation is enabled. Animation displays the QN-MHP layout and entity movement, but it also slows the run because many GUI updates occur.

4. File-by-file reference

File	Primary role	What it does in practice
run_code.py	Program entry point	Starts the GUI, builds the environment and model, launches entity generation, runs to `SIMTIME`, shows plots, and prints departure summaries.
gui_general.py	Top-level GUI workflow	Contains the main menu, task-definition screen, simulation-parameter screen, and entity-definition / parameter-launch screen.
gui_basic_BE.py	Basic-BE parameter GUIs	Contains one class per configurable basic BE and saves values into files.
gui_compound_BE.py	Compound-BE parameter GUIs	Contains the configuration windows for search, tracking, and tracing tasks. Values are saved per task.
entity_basic_BE.py	Basic entity generators	Generates color or text entities with arrival schedule, type, and attribute payloads.
entity_compound_BE.py	Compound task generators	Creates the initial entity and attributes for compound tasks and starts the corresponding process in `compound_BE.py`.
main_process.py	Task dispatcher	Reads the ordered BE sequence for the current task and invokes each BE process one by one.

basic_BE.py	Basic behavior implementation	Implements perception, judgment, response, arithmetic, look-at, and count behaviors.
compound_BE.py	Compound behavior implementation	Implements recursive search plus tracking and tracing loops, including RMSE updates.
model.py	QN-MHP resource model	Defines SimPy resources for servers 1-8, A/B/C/F/W/Y/Z, hands, and eyes. Also defines server service functions.
animation_general.py	Entity animation helper	Creates circles/text tags and animates entry, add, leave, and delete operations on the QN-MHP canvas.
qn_mhp_layout.py	Canvas and layout drawing	Provides scalable canvas classes and the background/server layout used for the animation view.
display_basic_BE.py	Basic reaction displays	Contains task-specific windows for button reactions, counting, arithmetic, look-at table, and click-space displays.
display_compound_BE.py	Compound reaction displays	Contains all tracking/tracing display classes and handles target/point motion and coordinate conversion.
plot.py	Result plotting	Shows mean/SD sojourn plots and RMSE plots after simulation.
utility.py	Shared constants and persistence helpers	Provides save/load helpers, color and text lists, track/tracing dictionaries, conversion helpers, and formatted summaries.

5. Basic BE parameter reference

BE	GUI class	Parameter	Allowed values	Meaning
Choice	GUI_BE_Choice	Choice number `N`	Integer 1-6	Number of response options used by choice-dependent windows and logic.
Mouse_click	GUI_BE_Mouseclick	Mouse click dimension	1 / 2 / 3	Sets whether the click display is 1D, 2D, or 3D.
Look_at	GUI_BE_Lookat	Eye location x and y	Each integer 1-9	Discrete table location used for fixation.

Judge_identity	GUI_BE_Judgei	Identity type and target value	Identity = Text or Color; value depends on selected identity	Defines what counts as a match.
Count	GUI_BE_Count	Count length	Integer 1-10	Number range length used by the counting task.
Cal_single_digit_num	GUI_BE_Cal_single_digit_num	Arithmetic operation	Add / Subtract / Multiplication / Division	Defines the operation used in arithmetic trials.

6. Compound BE parameter reference

Compound BE	Parameters shown in GUI	Allowed values / ranges	Meaning
Look_for	Target identity	Identity = Text or Color; value chosen from `look_for_ls`	Defines the search target stored in `judgei_target_dic.txt`.
Tracking_1D	Cursor start x, target start x, freq, amp, response	x in 0-100; freq = Slow/Medium/Quick; amp = Small/Medium/Large; response = Click mouse/Press keyboard	Bounded 1D tracking.
Tracking_1D_1W	Cursor start x, target start x, freq, amp, response	Same ranges as Tracking_1D	One-way 1D tracking.
Tracking_1D_Random	Cursor start x, target start x, freq, amp, response	Same ranges as Tracking_1D	Random 1D tracking.
Tracking_2D	Cursor start x, target start x, freq, amp, response, shape	x and y in 0-100 where applicable; shape selected in GUI	Bounded 2D tracking.
Tracking_2D_Random	Cursor start x/y, target start x/y, freq, amp, response	All coordinates in 0-100	Random 2D tracking.
Tracking_2D_1W	Cursor start x/y, target start x/y, freq, amp, response, shape	All coordinates in 0-100	One-way 2D tracking.
Tracing_1D_Bounded	Point start x, direction, x amplitude, response	x in 0-100; amplitude = Small/Medium/Large	Bounded 1D tracing.
Tracing_1D_1W	Point start x, x amplitude, response	x in 0-100	One-way 1D tracing.
Tracing_2D_Bounded	Point start x, x amp, y amp, shape, direction, response	x in 0-100; amplitudes from GUI options	Bounded 2D tracing.

5A. Current-version configuration and entity validation

This section supplements the Basic BE parameter reference. The original reference tables remain available on the preceding pages; the instructions below describe the newer configuration behaviors.

Count and Cal_single_digit_num

Both behaviors require a **Text** entity. When configuring a See or Hear entity for a task that contains Count or Cal_single_digit_num, choose Text rather than Color. A Color selection is rejected with a warning; do not try to supply arithmetic or counting values through Color entities.

For **Count**, configure Count Length in the Count parameter window. The incoming Text entity supplies the numeric content needed for the counting sequence. For **Cal_single_digit_num**, select the arithmetic operation in its parameter dialog and use Text entities carrying numerical instructions.

Choice, Mouse_click, and Look_at

Choice specifies the number of response alternatives. Mouse_click offers 1D, 2D, and 3D coordinate modes; choose the mode that corresponds to the task geometry. Look_at uses a visual target position and a current gaze/fixation position; the eye-movement timing model is described below.

For every entity row, specify First Arrival Time, Inter-arrival Time (IAT), and Occurrences. A row with incomplete numerical fields should be completed or cleared. Keep all arrival and timing values in milliseconds.

Task ordering and concurrency

Define and save the task sequence before configuring entity parameters. Multiple behavior elements may be configured within the task matrix, but execution order is determined by the assigned order values. The current documented configuration supports at most two simultaneously running tasks.

Tracing_2D_1W	Point start x, x amp, y amp, shape, response	x in 0-100; amplitudes from GUI options	One-way 2D tracing.

7. How each basic BE operates

Basic BE	Purpose
See	Perceptual processing of visual entities.
Hear	Perceptual processing of auditory entities.
Store_to_WM	Store relevant information in working memory.
Choice	Select one alternative or response.
Judge_identity	Compare the current entity with a target identity.
Judge_relative_location	Compare relative position of target and cursor/point.
Cal_single_digit_num	Single-digit arithmetic operation.
Press_button	Keyboard/button response task.
Mouse_click	Mouse click response in 1D/2D/3D.
Look_at	Move fixation to a discrete location.
Count	Counts over a user-defined length.

8. How each compound BE operates

Compound BE	Purpose
Look_for	Visual search (by text or by color) across the predefined item list. Simulates the human visual search process in a 3 × 3 grid. Four items with different color and text identities are generated and randomly assigned to positions in the grid. At each search step, the program randomly locates one position and compares the identity of the item at that position with the user-defined target identity. If the identity matches, the visual search ends. Otherwise, the search continues until the target is found or the simulation time is reached.
Tracking_1D	Bounded 1D target tracking. The target point moves in one direction according to the user-defined frequency and amplitude. Once it reaches the 1D boundary, it reverses direction. Simulates the process in which a human moves a cursor point, either by

	clicking to a new position or by using the arrow keys, to make the cursor point as close as possible to the target point.
Tracking_1D_1W	One-way 1D tracking. The target point continuously moves in a single direction according to the user-defined frequency and amplitude. There is no boundary, so the target point keeps moving in the same direction. Simulates the process in which a human moves a cursor point, either by clicking to a new position or by using the arrow keys, to make the cursor point as close as possible to the target point.
Tracking_1D_Random	Random-direction 1D tracking. The target point moves randomly to the left or right according to the user-defined frequency and amplitude. Simulates the process in which a human moves a cursor point, either by clicking to a new position or by using the arrow keys, to make the cursor point as close as possible to the target point.
Tracking_2D	Bounded 2D tracking. The target point moves in both the x and y directions according to the user-defined frequency and amplitude. Once it reaches the boundary in either dimension, it reverses direction within that dimension. Simulates the process in which a human moves a cursor point, either by clicking to a new position or by using the arrow keys, to make the cursor point as close as possible to the target point.
Tracking_2D_Random	Random 2D tracking. The target point moves randomly in both the x and y directions according to the user-defined frequency and amplitude, with direction independently changing at each step. Simulates the process in which a human moves a cursor point, either by clicking to a new position or by using the arrow keys, to make the cursor point as close as possible to the target point.
Tracking_2D_1W	One-way 2D tracking. The target point continuously moves in a fixed direction along the x-axis according to the user-defined frequency and amplitude, while the corresponding y-position follows the user-defined trajectory. There is no boundary, so the target point keeps moving in the same direction. Simulates the process in which a human moves a cursor point, either by clicking to a new position or by using the arrow keys, to make the cursor point as close as possible to the target point.
Tracing_1D_Bounded	Bounded 1D tracing. Simulates the process in which the user controls a point either by clicking to move it to a new position or by using the arrow keys to move it along a 1D trajectory. Once it reaches the 1D boundary, it reverses direction. For keyboard control, the movement distance produced by each key press depends on the user-defined amplitude. For mouse-click control, the movement distance is determined by a default fixed distance defined in the program.
Tracing_1D_1W	One-way 1D tracing.

	Simulates the process in which the user controls a point, either by clicking to move it to a new position or by using the arrow keys to move it along a 1D trajectory. The point continuously progresses in one direction, and there is no boundary. For keyboard control, the movement distance produced by each key press depends on the user-defined amplitude. For mouse-click control, the movement distance is determined by a default fixed distance defined in the program.
Tracing_2D_Bounded	Bounded 2D tracing. Simulates the process in which the user controls a point, either by clicking to move it to a new position or by using the arrow keys to move it along a 2D trajectory. The point moves within predefined boundaries, and when it reaches a boundary in either dimension, it reverses direction in that dimension. For keyboard control, the movement distance produced by each key press depends on the user-defined amplitude ((on both x and y axis)). For mouse-click control, the movement distance is determined by a default fixed distance defined in the program.
Tracing_2D_1W	One-way 2D tracing. Simulates the process in which the user controls a point, either by clicking to move it to a new position or by using the arrow keys to move it along a 2D trajectory. The point continuously progresses in one direction. For keyboard control, the movement distance produced by each key press depends on the user-defined amplitude (on both x and y axis). For mouse-click control, the movement distance is determined by a default fixed distance defined in the program.

9. Displays and plotting

`display_basic_BE.py` contains task-facing windows for button response, counting, arithmetic, the look-at grid, and click-space visualization.

`display_compound_BE.py` contains all tracking/tracing windows and the motion logic for targets and points. These classes use Matplotlib embedded inside Tkinter windows.

`plot.py` produces two types of output: a sojourn-time plot and an RMSE plot. When a tracking or tracing task exists, RMSE is shown.

10. Timing, units

All process times and comparisons in the core logic use SimPy time units, and the code labels them in milliseconds. These are simulation times and modeled real-world times, not real-world elapsed computer time.

In tracking and tracing tasks, positions are first updated in the program's internal coordinate space based on the user-defined amplitude and frequency. These positions are then displayed on the screen in pixels and converted into physical distance units (mm). Therefore, the movement itself is defined in coordinate units, but performance measures such as RMSE are ultimately calculated in millimeters. This design allows the model to remain independent of screen resolution while ensuring that performance metrics are interpretable and comparable in real-world units.

8A. Visual attention and motor timing in the current version

Look_at: fixation and target

The Look_at reaction display distinguishes the **current fixation** from the **target**. A blue marker indicates the current fixation, and a red marker indicates the visual target. After the modeled eye movement, fixation updates to the new target. The gaze transition is handled by the eye-movement model in addition to the behavior-element processing stages.

Look_at targets are specified using physical screen positions in millimeters. Internal normalized positions are used where required for display/model integration. A position on a screen of width W mm and height H mm can be mapped as x_norm = 100 x_mm / W and y_norm = 100 y_mm / H; the reverse mapping multiplies the normalized fraction by the relevant screen dimension. Check the application coordinate origin and axis direction when interpreting any particular GUI.

Look_for: visual search

Look_for is a separate visual-search task. Its reaction display presents multiple candidate items; the model compares the selected item identity with the configured search target and continues until a match or the simulation limit. Its multi-target display should not be confused with the two-marker Look_at fixation/target display.

Eye Server and saccade duration

The Eye Server uses the configured screen width, height, and viewing distance (all in mm) to obtain the visual angle between fixation and target, then calculates eye-movement duration in milliseconds. The geometry is physical; screen-pixel values alone should not be interpreted as millimeters. Eye-movement timing is part of simulated time, not the elapsed computer runtime.

Mouse_click timing

Mouse movement time depends on the target distance A and target width W, using a Shannon-form Fitts-law index of difficulty: ID = log2(1 + A/W). The resulting duration is computed from the configured model coefficients. Target width must be positive, and both distance and width must be expressed in compatible units.

11. Practical troubleshooting notes

- Always define tasks before opening entity or BE parameter windows. Those windows depend on task structure already existing.
- If a parameter window looks empty or incorrect, verify that the related BE is actually present in the task-definition matrix and that the task was saved.
- If you want a reusable configuration, use `save as` before starting a completely new run, because the default `backup/` folder is reset at startup.
- If animation or tracking windows cause the run to feel slow, disable animation or reduce the total simulation time while debugging.
- Entity rows with partially filled numeric fields trigger validation errors. Either leave the row fully blank or fill all required numeric fields.
- If the program reports an unexpected error or the GUI becomes unresponsive after a failed run, restart the Python console or kernel before running the program again. This helps clear any residual variables, open windows, or incomplete simulation states from the previous run.
- System environment note: The current version of the software has been tested on Windows 10 and was developed in Python 3.9. For best compatibility, it is recommended to run the program in the same environment. Required packages include simpy, numpy, matplotlib, tkinter, and pyautogui. The program also uses standard Python libraries such as os, sys, time, math, random, pickle, shutil, stat, datetime, and atexit.

12. An Example

12.1 example overview

This example shows how to use this QN-MHP Python to simulate a multitasking situation involving two tasks. The first task is a simple or choice reaction time task. When a human sees an entity, they process its color and press the corresponding button with the same color from a set of choices. The second task is a counting task. When the human hears an instruction to count within a specified range, for example from 3 to 9, they are required to count aloud in sequence, such as 3, 4, 5, 6, 7, 8, and 9.

12.2 start the program

Open a terminal in the project directory and run `run_code.py`.

10A. Coordinate representations and result interpretation

Physical screen coordinates and normalized positions

Look_at and Look_for define visual target positions with physical screen coordinates (mm). Internal normalized positions may be represented on a 0-100 scale. To convert a normalized x coordinate to millimeters, use x_mm = (x_norm / 100) x screen_width_mm; likewise y_mm = (y_norm / 100) x screen_height_mm. The inverse operation converts physical millimeters to normalized positions.

Example: on a 500-mm-wide screen, x_norm = 40 corresponds to x_mm = 200 mm. This conversion describes spatial position, not visual angle. The eye-movement model also requires the viewing distance to determine angular displacement.

Tracking and Tracing coordinates

Tracking and Tracing use task-specific plotting coordinates. The simulation updates target and cursor/point positions in that coordinate system, maps them to screen positions for visualization, and converts displacement to physical units when calculating performance metrics such as RMSE. Do not assume that a plot coordinate, normalized coordinate, screen pixel, and physical millimeter are interchangeable.

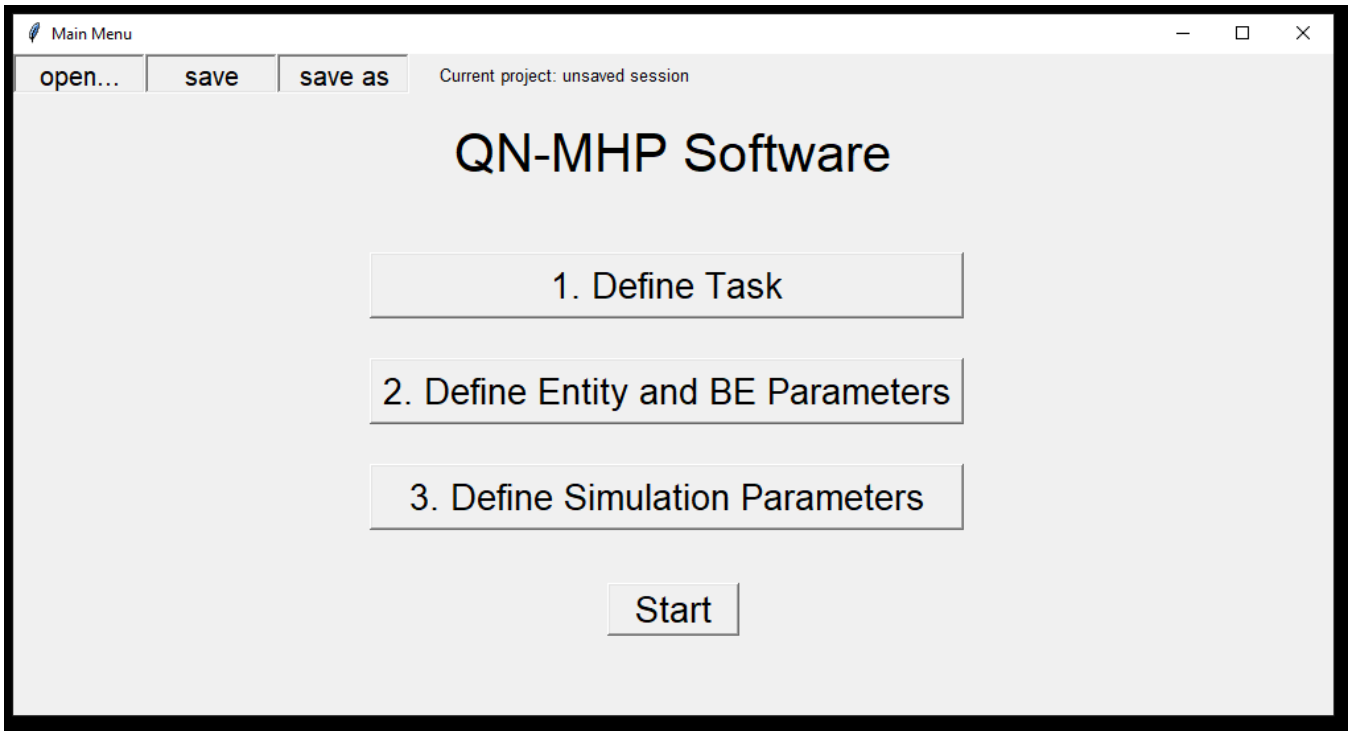
Simulation time, plots, and logs

Simulation time is measured in milliseconds. Plots summarize the simulated entity sojourn times and, where applicable, tracking/tracing error. The timestamped work log records server activity and departures. Animation may increase wall-clock runtime without changing the meaning of the simulated time values.

Compatibility and saved configurations

The original screenshots in the following worked example were captured on Windows. Window appearance may differ on macOS, but the example task sequence and parameter-setting logic remain instructive. Use Save As to preserve a reusable configuration before ending a session; temporary backup data is not intended for permanent storage.

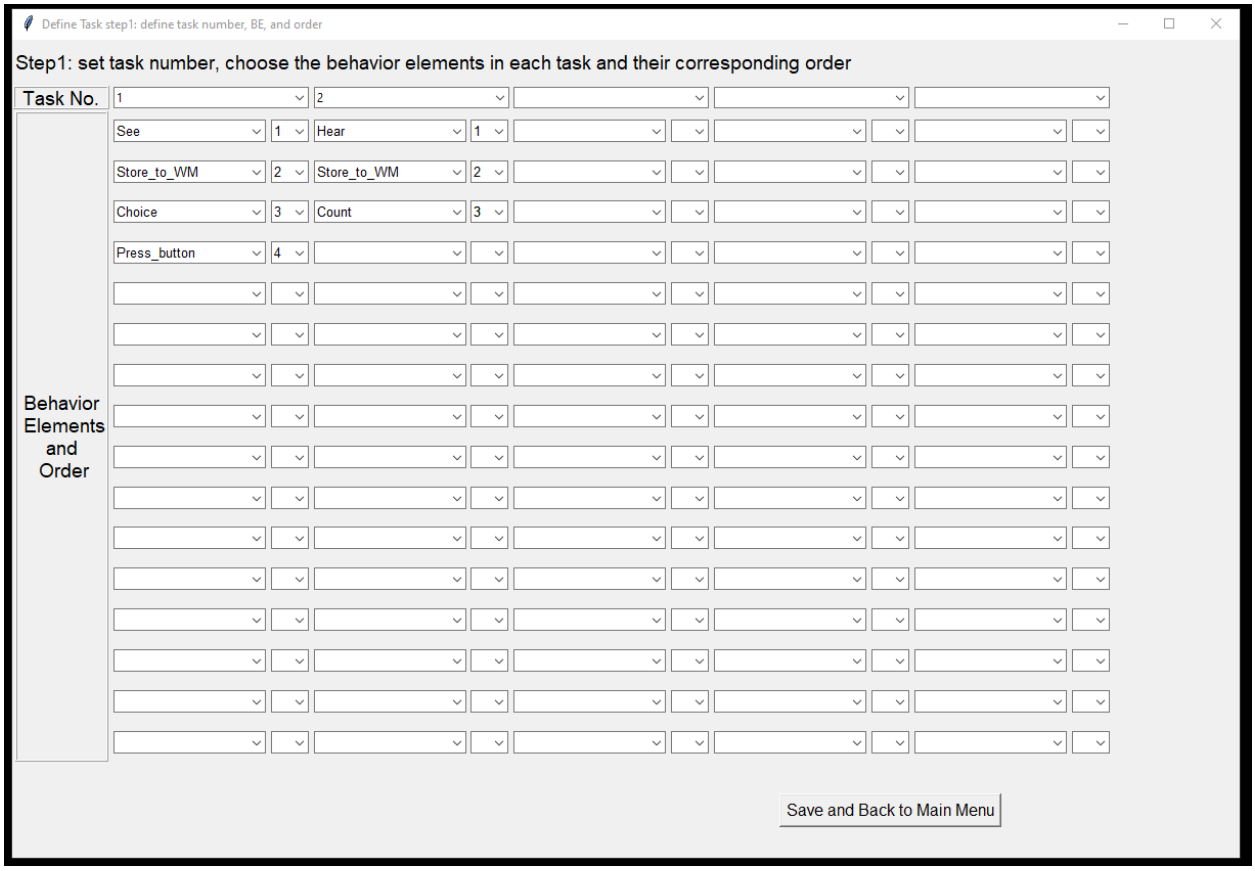
A main menu will be shown, as illustrated below:



Save your project folder to a local location or open an existing project folder before you begin. Otherwise, your inputs will be stored only in the temporary backup/ folder in the current working directory, and they will be deleted the next time the program starts.

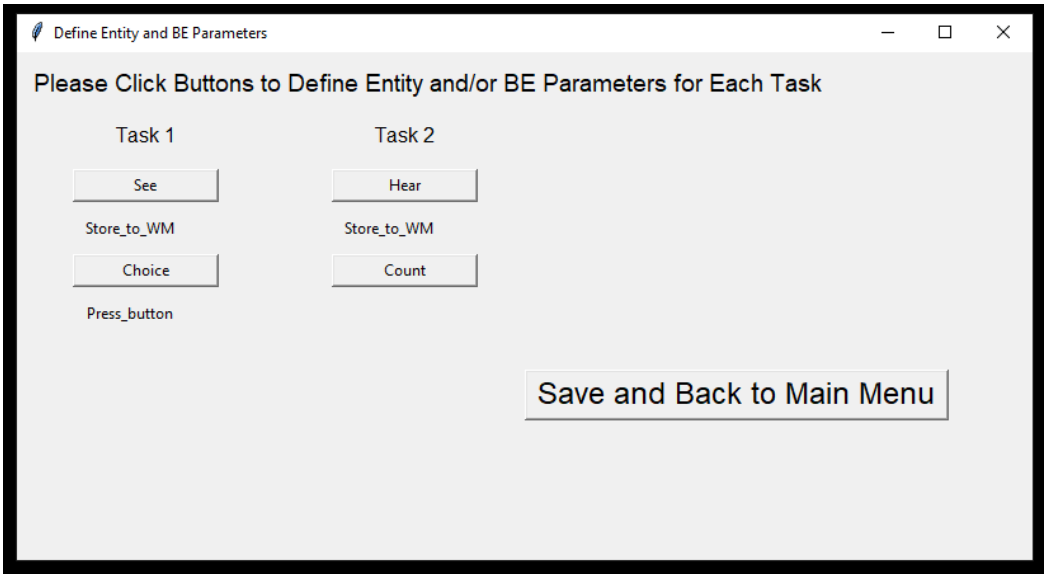
12.3 Define task

Click the "1. Define Task" button on the main menu, then select the task sequence and BE sequence for each task according to your task requirements. Note that the current version supports only up to two concurrent tasks. Then click "Save and Back to Main Menu" button to return to the main menu.



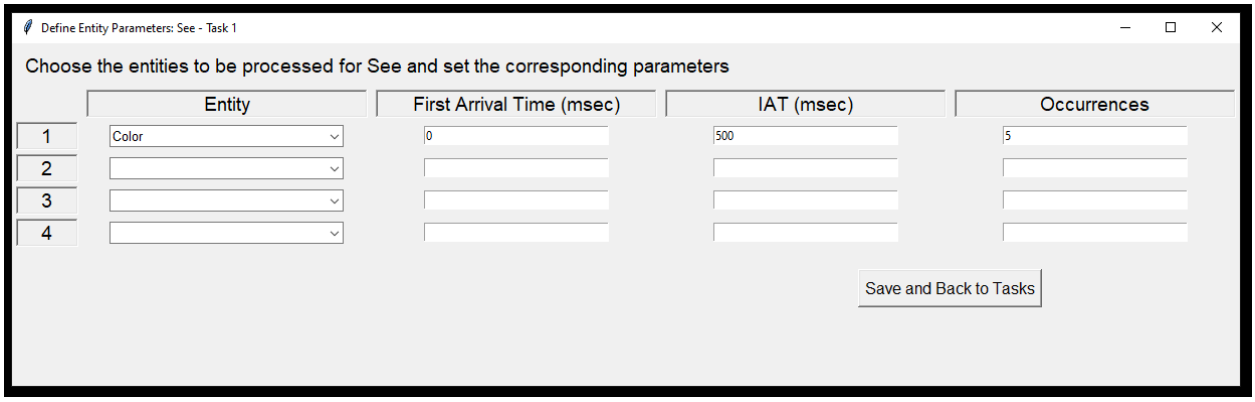
12.4 Define entity and BE parameters

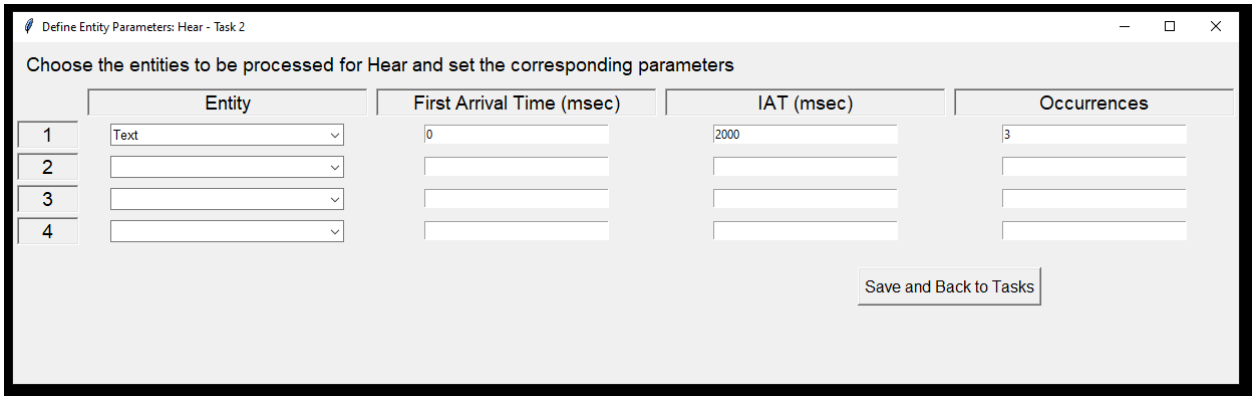
Click the "2. Define Entity and BE Parameters" button on the main menu. Press each clickable button under each task to set entity or BE parameters.



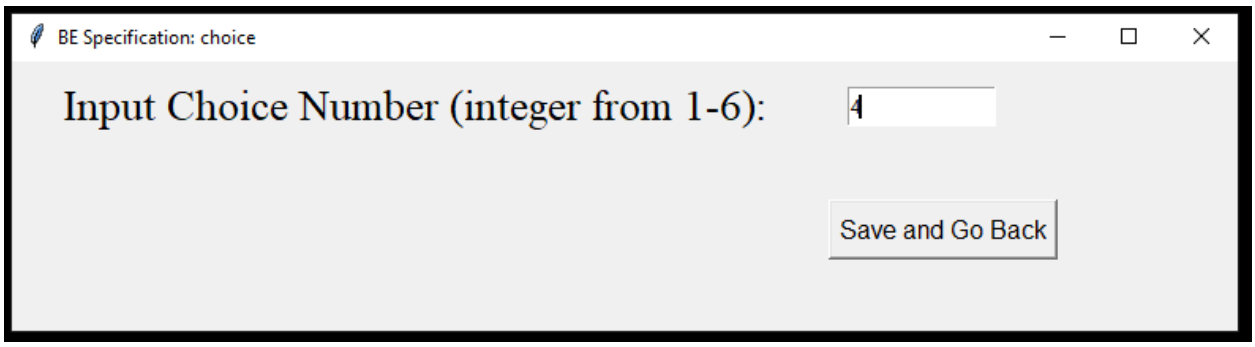
Entities represent different information signals from the external world. When defining entity parameters for See or Hear, first select the entity type. You can choose either "color" or "text." In this example, for the choice reaction time task, either color or text can be selected because the response can be based on either feature. However, for the counting task, color does not apply, because the counting instruction can only be represented as text information.

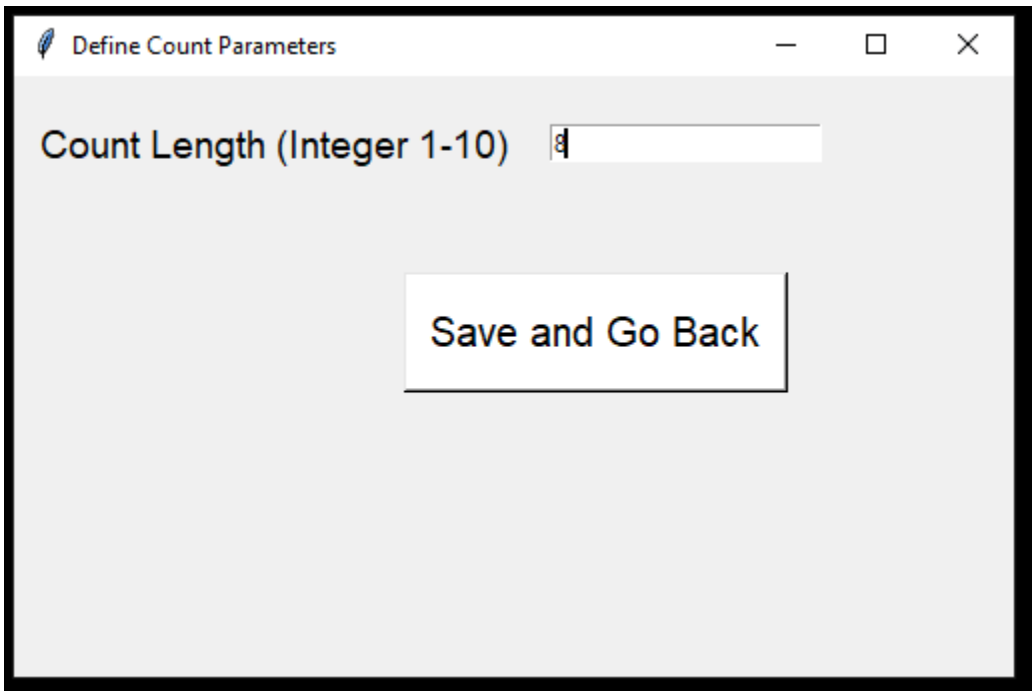
Next, specify the first arrival time for this type of entity. IAT stands for inter-arrival time, which indicates the time interval between consecutive entities. Occurrences specifies how many entities of this type will be generated.





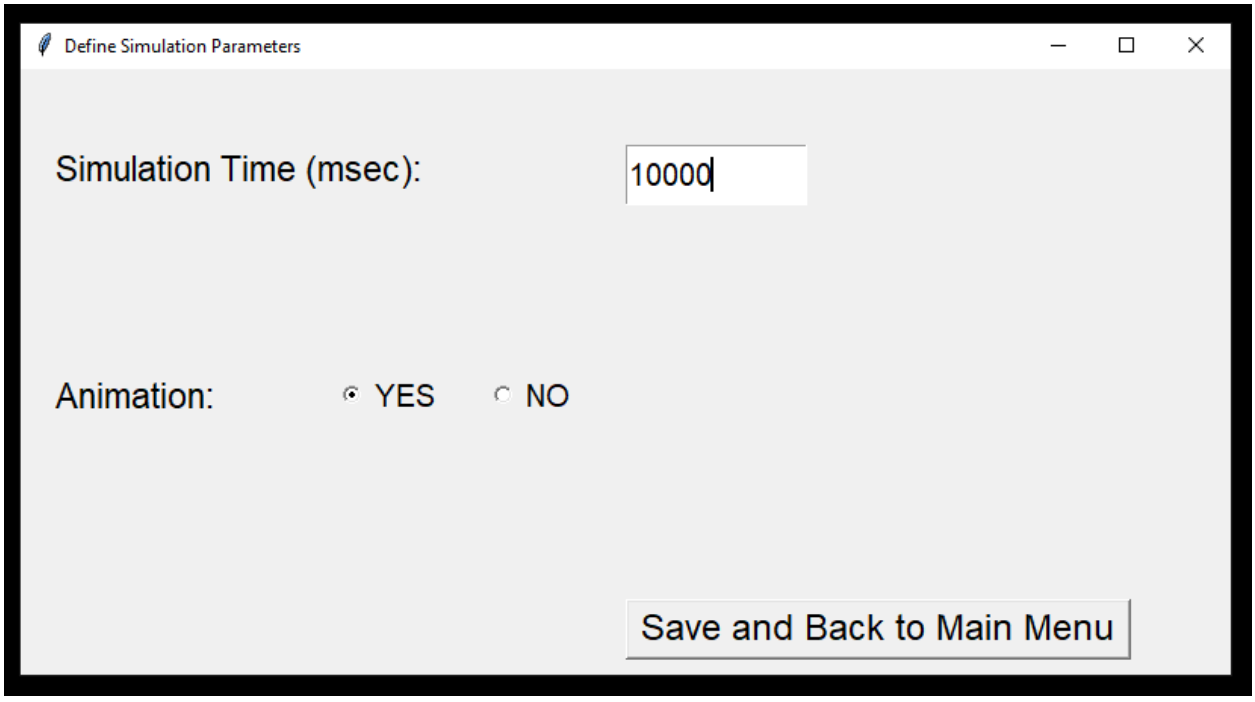
Input the BE parameters according to the prompts and your task design. The current version includes some parameter limitations to ensure stable operation and clear display presentation. Future developers may extend these settings to better match specific task requirements.





12.5 Define simulation parameters

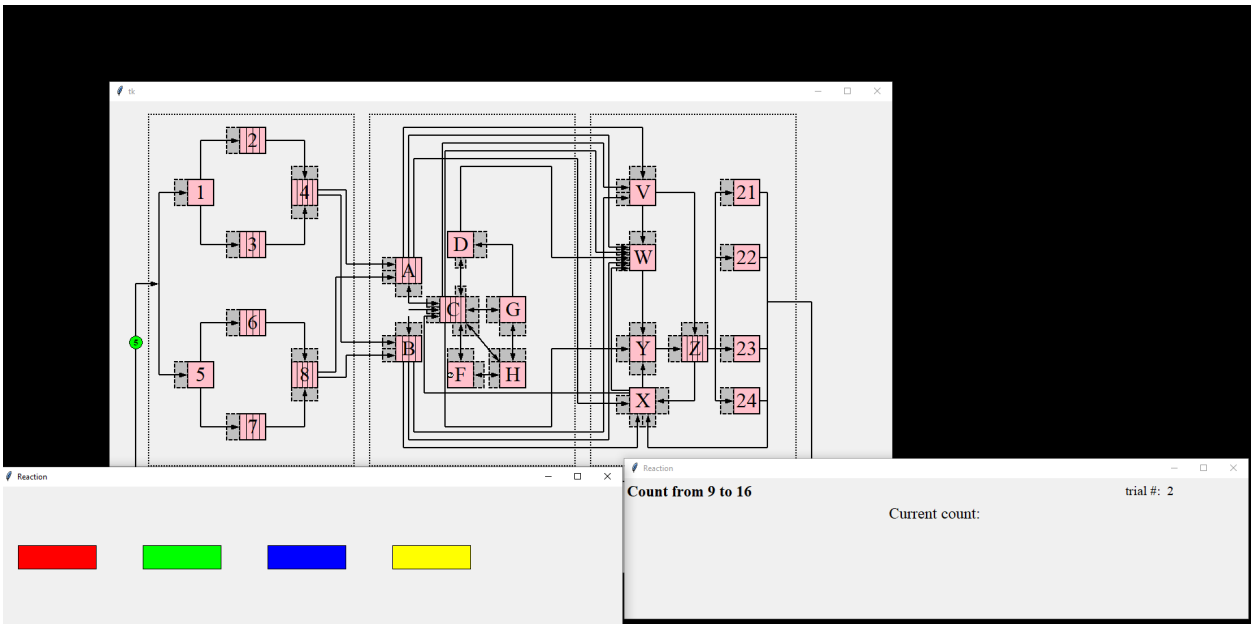
Click the "3. Define Simulation Parameters" button on the main menu, then enter the total simulation time and select whether you want to display the animation.



12.6 Animation and reaction displays

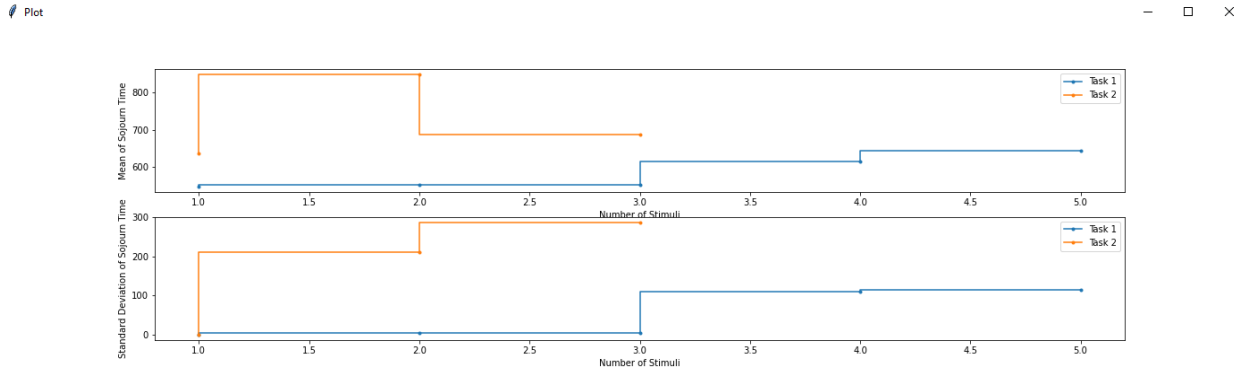
During the simulation, the animation shows how each entity, carrying different color and text information, travels through the QN-MHP network and is processed by different servers. Each grid inside a server represents its processing capacity, and the grey area represents the queue.

For the color choice reaction time task, the reaction display simulates the human response by pressing the corresponding button with the same color as the incoming entity. The counting reaction display shows the process of completing the counting task.



12.7 Plot and log

After the simulation, a plot displays the mean and standard deviation of reaction time after each entity (i.e., each trial) completes its processing. A work log is also automatically generated using the computer time as the file name and is saved in the working path. This log records the timestamp of each server activity, as well as the entity departure number for each server in each task.



13. Developer Guide

This chapter is intended for developers who want to modify existing behavior elements (BEs), add task types, adjust the queueing network, or reuse the simulation logic in a new application. The GUI workflow and original worked example in Sections 1–12 remain applicable. The file names and function names below are taken from the current Python project.

13.1 Understand the execution flow

1. Configuration: `gui_general.py` builds the main menu and task sequence. `gui_basic_BE.py` and `gui_compound_BE.py` collect BE-specific parameters. `utility.py` supports persistence and shared configuration.

2. Entity generation: `entity_basic_BE.py` generates basic visual/auditory entities; `entity_compound_BE.py` prepares compound-task entities. Each entity carries an attribute dictionary that later BEs read and update.

3. Execution: `run_code.py` initializes the SimPy environment. `main_process.py` dispatches configured BE functions in task order. Basic and compound behavior implementations live in `basic_BE.py` and `compound_BE.py`.

4. Resources and output: `model.py` defines server resources and service functions. Display files provide task-facing windows; `plot.py` summarizes results. Console output is also written to timestamped files under `log/`.

13.2 Modify an existing Basic BE

Start by locating the BE function in `basic_BE.py`. Follow the entity attributes it reads, the server resources it requests, and any GUI values loaded through the shared configuration. Avoid changing the function signature without updating every caller.

```
def BE_name(qn_mhp, env, i, j, k, attribute,
            outserver_dic, arrival_time, generation, ppt, cpt, mpt):
    # Existing BE functions are SimPy generator processes.
    # Preserve yield / env.process behavior and entity attributes.
```

If a parameter changes, update the corresponding GUI dialog and its validation in `gui_basic_BE.py`, plus any persistence and entity-generation logic that depends on that parameter. For example, Count and Cal_single_digit_num require Text entities; reject unsupported Color configurations before the simulation starts rather than inventing missing numeric attributes.

13.3 Add a new Basic BE

Step 1. Implement a SimPy-compatible BE function in `basic_BE.py` using the existing BE function signature. Follow an existing function as a template for server acquisition, entity handling, and departure logging.

Step 2. Add the function import and its exact display name to `BE_FUNCTIONS` in `main_process.py`. The dispatcher resolves BE names with `get_be_function(name)`; an unregistered name raises a ValueError.

```
from basic_BE import My_new_BE
BE_FUNCTIONS["My_new_BE"] = My_new_BE
```

Step 3. Add the same BE name to the task-selection options in `gui_general.py`. If user inputs are required, add a parameter dialog to `gui_basic_BE.py` and connect it to the parameter launcher. Ensure its values can be saved and reopened.

Step 4. Ensure the entity generator supplies every attribute the new BE expects. If it needs a new reaction window, add the display to `display_basic_BE.py`. Run a minimal single-BE task before combining it with others.

13.4 Create or extend a Compound BE

Compound behavior logic belongs in `compound_BE.py`, with configuration in `gui_compound_BE.py` and entity setup in `entity_compound_BE.py`. Register the new name in the task-selection GUI and `main_process.py`. Reuse existing

basic BEs where appropriate, but explicitly define entity lifecycle, stopping conditions, and how intermediate data are passed between steps.

For tracking and tracing, inspect `display_compound_BE.py` and verify coordinate conversion before computing error metrics. Internal task coordinates, screen pixels, physical millimeters, and eye-model normalized coordinates are not interchangeable.

13.5 Change server resources or processing times

Review `QN_MHP.resource()` and `server_def` in `model.py`. Server methods include `server1` through `server8` and named servers such as A/B/C/F/W/Y/Z. Their processing calls use the simulation environment and timing parameters (`ppt`, `cpt`, `mpt`).

When changing a capacity, delay, or route, check both the service function and the BE that requests it. Resource contention is part of the queueing model: modifying capacity can change waiting time and the behavior of concurrent tasks. Keep all delays nonnegative and in simulated milliseconds. Verify the server departure summary after every structural change.

13.6 Eye movement and mouse movement models

Eye: `eye_movement_model.py` defines initial fixation, shared fixation state, display geometry, visual-angle calculation, and saccade duration. Screen width, height, and viewing distance are expressed in millimeters; shared geometry is read from `visual_search_layout.json`. `angular_distance_deg(start, target)` uses normalized 0–100 positions and returns degrees; `calculate_saccade_duration(amplitude_deg)` returns milliseconds.

The intercept, slope, noise, and initial-fixation settings are model assumptions, not validated participant-specific coefficients. For experimental use, calibrate them and document the source of each parameter. Look At and Look For may share the eye resource; preserve synchronization when updating fixation.

Mouse: `mouse_movement_model.py` exposes `calculate_mouse_movement_time(distance, target_width, model=None)`. The default is Shannon Fitts law: `a + b log2(1 + distance / width)`. Distance and width must use the same spatial units; output is milliseconds. The function validates input and prevents negative durations. Set `FITTS_INTERCEPT_MS` and `FITTS_SLOPE_MS` only after calibration.

13.7 Testing, debugging, and reproducibility

Recommended test order: (1) start with one task and one entity; (2) check the generated attribute dictionary; (3) confirm each BE enters and leaves the expected servers; (4) compare simulated timing and the work log; (5) enable reaction displays; (6) test two concurrent tasks and verify shared-resource behavior; (7) save the configuration and reopen it.

Useful checks include `python -m compileall .` for syntax, a short simulation time for faster iteration, and comparison of `log/` output before and after a change. GUI screenshots from the original Windows example remain useful for task design, although window styling can differ on macOS.

Common issues: **KeyError** usually indicates a missing entity attribute or incompatible entity type; **negative timeout** indicates an invalid duration formula; **empty results** can mean no entity was generated or a task never reached its final server; **unexpected eye movement** often indicates inconsistent coordinate systems or stale fixation state. Do not suppress these errors without addressing the source.

13.8 Developer checklist before distributing a modified version

Confirm that GUI choices match the dispatcher registry; parameter dialogs validate inputs; entity generators provide required attributes; all SimPy timeouts are finite and nonnegative; coordinate units are documented; Save As/Open still work; output plots and logs are correct; and both single-task and concurrent-task examples complete. Keep the original worked example and screenshots as reference material, and update any instructions that no longer match the GUI.